Machine Learning Lab

Yukun Jiao

2025-09-03

Lab 3

Part 1:

1.

```
library(keras); library(keras3); library(tensorflow)
library(ggplot2); library(dplyr); library(tidyr)
library(data.table); library(rpart); library(caret)
library(nnet)
```

```
adults <- readRDS("~/Documents/ML/Labs/Lab3/adults.rds")
adults <- na.omit(adults)
# X_all <- scale(adults[, -which(colnames(adults) == "y")])
X_all <- adults[, -which(colnames(adults) == "y")]
# X_all <- scale(adults[, -6])
y_all <- adults[, "y"]       # labels (0/1)

set.seed(42)
n <- nrow(X_all)
idx <- sample(seq_len(n), size = floor(0.8 * n))
X_train <- X_all[idx, , drop = FALSE]
y_train <- y_all[idx]
X_test  <- X_all[-idx, , drop = FALSE]
y_test  <- y_all[-idx]
```

Based on the information that there are no complex interactions or non-linearities, it can be inferred that a neural network will not excel on this dataset. Neural networks are better

suited for capturing non-linear relationships and interactions. Although they can also capture linear relationships, they require more parameters, lack interpretability, and their predictive performance is likely to be close to that of a simple linear regression model.

2.

```
set.seed(1984)
train_df <- data.frame(y = y_train, X_train)
test_df <- data.frame(y = y_test, X_test)

salary_glm <- glm(y ~ ., data = train_df, family = "binomial")

p_hat <- predict(salary_glm, newdata = test_df, type = "response")


acc_bin <- function(p, y_true, thr = 0.5) mean((p > thr) == y_true)
acc_bin(p_hat, test_df$y, thr = 0.5)
```

```
[1] 0.807964
```

```
# y_pred <- ifelse(p_hat >= 0.5, 1, 0)
# accuracy <- mean(y_pred == test_df$y)
# accuracy
```

The accuracy is 0.807964.

3.

```
input_dim <- ncol(X_train)

k_clear_session()
model_A <- keras_model_sequential() |>
  # Hidden layer:
  # units=32 (width of layer), activation="relu" (nonlinearity)
  layer_dense(units = 5, activation = "relu", input_shape = input_dim) |>
  # Output layer for binary classification:
  layer_dense(units = 1, activation = "sigmoid")
```

```r
model_A |>
  compile(
    optimizer = optimizer_adam(1e-3),   # optimizer/learning rate
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_A <- model_A |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
  )

proba_A <- as.numeric(model_A |> predict(X_test, verbose = 0))
acc_A   <- acc_bin(proba_A, y_test)
cat(sprintf("[adults] Model 1 (1 hidden) - test accuracy: %.3f\n", acc_A))
```

```
[adults] Model 1 (1 hidden) - test accuracy: 0.817
```

The accuracy of the MNN is slightly better compared to a standard logistic regression model, but still very close. Moreover, it requires more computational power and parameter tuning, and it lacks interpretability. This is consistent with my assessment in #1.

4.

```r
k_clear_session()
model_D <- keras_model_sequential() |>
  layer_dense(units = 256, activation = "relu", input_shape = input_dim) |>
  layer_dense(units = 128, activation = "relu") |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 1, activation = "sigmoid")

model_D |>
  compile(
    optimizer = optimizer_adam(1e-3),
```

```
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_D <- model_D |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
  )

proba_D <- as.numeric(model_D |> predict(X_test, verbose = 0))
acc_D   <- acc_bin(proba_D, y_test)
cat(sprintf("[adults] Model 2 (4 hidden) - test accuracy: %.3f\n", acc_D))
```

```
[adults] Model 2 (4 hidden) - test accuracy: 0.826
```

```
# model_D |> summary()
```

The number of trainable parameters is $1536 + 32896 + 8256 + 2080 + 33 = 44,801$.

The performance is better, but the improvement is not very large. I suspect there may be some limited non-linear relationships and interactions that allow the neural network to capture them, leading to the improvement in accuracy.

5.

```
adults_aug <- readRDS("~/Documents/ML/Labs/Lab3/adults_aug.rds")
adults_aug <- na.omit(adults_aug)
X_all <- adults_aug[, -which(colnames(adults_aug) == "y")]
y_all <- adults_aug[, "y"]       # labels (0/1)

set.seed(42)
n <- nrow(X_all)
idx <- sample(seq_len(n), size = floor(0.8 * n))
X_train <- X_all[idx, , drop = FALSE]
y_train <- y_all[idx]
```

```r
X_test  <- X_all[-idx, , drop = FALSE]
y_test  <- y_all[-idx]
```

```r
set.seed(1984)
train_df <- data.frame(y = y_train, X_train)
test_df <- data.frame(y = y_test, X_test)

salary_glm <- glm(y ~ ., data = train_df, family = "binomial")

p_hat <- predict(salary_glm, newdata = test_df, type = "response")


acc_bin <- function(p, y_true, thr = 0.5) mean((p > thr) == y_true)
acc_bin(p_hat, test_df$y, thr = 0.5)
```

```
[1] 0.8074521
```

```r
input_dim <- ncol(X_train)

k_clear_session()
model_A <- keras_model_sequential() |>
  # Hidden layer:
  # units=32 (width of layer), activation="relu" (nonlinearity)
  layer_dense(units = 5, activation = "relu", input_shape = input_dim) |>
  # Output layer for binary classification:
  layer_dense(units = 1, activation = "sigmoid")

model_A |>
  compile(
    optimizer = optimizer_adam(1e-3),   # optimizer/learning rate
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_A <- model_A |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
```

```
  )

proba_A <- as.numeric(model_A |> predict(X_test, verbose = 0))
acc_A   <- acc_bin(proba_A, y_test)
cat(sprintf("[adults_aug] Model 3 (1 hidden) - test accuracy: %.3f\n", acc_A))
```

```
[adults_aug] Model 3 (1 hidden) - test accuracy: 0.962
```

```
k_clear_session()
model_D <- keras_model_sequential() |>
  layer_dense(units = 256, activation = "relu", input_shape = input_dim) |>
  layer_dense(units = 128, activation = "relu") |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 1, activation = "sigmoid")

model_D |>
  compile(
    optimizer = optimizer_adam(1e-3),
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_D <- model_D |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
  )

proba_D <- as.numeric(model_D |> predict(X_test, verbose = 0))
acc_D   <- acc_bin(proba_D, y_test)
cat(sprintf("[adults_aug] Model 4 (4 hidden) - test accuracy: %.3f\n", acc_D))
```

```
[adults_aug] Model 4 (4 hidden) - test accuracy: 0.997
```

```
# model_D |> summary()
```

The neural network model should significantly outperform the standard logistic regression model.

On this augmented dataset, the neural network model substantially improves accuracy. This is because many variables containing non-linear relationships and interactions were added to the dataset. Neural networks can capture these effects very well, whereas the standard logistic regression model cannot, and therefore its accuracy is lower than that of the neural network model.

6.

```
set.seed(1984)

k_clear_session()
model_E <- keras_model_sequential() |>
  layer_dense(units = 256, activation = "relu", input_shape = input_dim) |>
  layer_dropout(rate = 0.1) |>
  layer_dense(units = 128, activation = "relu") |>
  layer_dropout(rate = 0.1) |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dropout(rate = 0.1) |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 1, activation = "sigmoid")

model_E |>
  compile(
    optimizer = optimizer_adam(1e-3),
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_E <- model_E |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
  )

proba_E <- as.numeric(model_E |> predict(X_test, verbose = 0))
```

```r
acc_E   <- acc_bin(proba_E, y_test)
cat(sprintf("[adults_aug] Model 5 (4 hidden) - test accuracy: %.3f\n", acc_E))
```

```
[adults_aug] Model 5 (4 hidden) - test accuracy: 0.997
```

```r
# model_E |> summary()

# results <- model_E |> evaluate(X_test, y_test, verbose = 0)
# results$accuracy
```

```r
k_clear_session()
model_F <- keras_model_sequential() |>
  layer_dense(units = 256, activation = "relu", input_shape = input_dim) |>
  layer_dropout(rate = 0.9) |>
  layer_dense(units = 128, activation = "relu") |>
  layer_dropout(rate = 0.9) |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dropout(rate = 0.9) |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 1, activation = "sigmoid")

model_F |>
  compile(
    optimizer = optimizer_adam(1e-3),
    loss = "binary_crossentropy",
    metrics = "accuracy"
  )

history_F <- model_F |>
  fit(
    X_train, y_train,
    epochs = 25,
    batch_size = 32,
    validation_split = 0.2,
    verbose = 0
  )

proba_F <- as.numeric(model_F |> predict(X_test, verbose = 0))
acc_F   <- acc_bin(proba_F, y_test)
cat(sprintf("[adults_aug] Model 6 (4 hidden) - test accuracy: %.3f\n", acc_F))
```

```
[adults_aug] Model 6 (4 hidden) - test accuracy: 0.242
```

```
# model_F |> summary()
```

A dropout rate of 0.1 performs better, possibly because dropout learning provides regularization and reduces variance. Although it may increase bias, the reduction in variance outweighs the increase in bias, which makes it worthwhile, and accuracy is further improved.

With a dropout rate of 0.9, accuracy drops dramatically. This is likely also due to reduced variance but with a much greater increase in bias. The increase in bias clearly outweighs the benefits of reduced variance, leading to a sharp decline in accuracy.

Part 2

1.

```
fashion_train <- readRDS("~/Documents/ML/Labs/Lab3/fashion_2016_train.rds")
fashion_test <- readRDS("~/Documents/ML/Labs/Lab3/fashion_2016_test.rds")
```

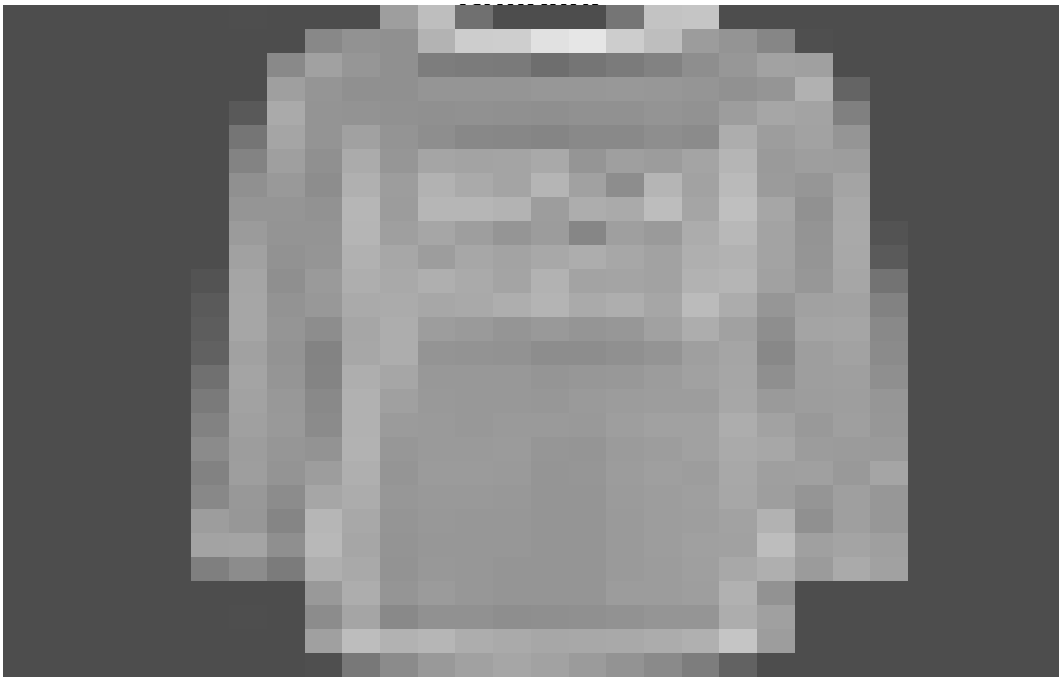
2.

take a look at these labels

```
x_train <- fashion_train$images
y_train <- fashion_train$labels
x_test <- fashion_test$images
y_test <- fashion_test$labels


# fashion_train$labels
num <- 5
# fashion_label <- fashion_train$labels[num]
# fashion_train$fashion_labels[fashion_label+1]

par(mar = c(0,0,0,0))
test <- x_train[num,,,]
item_name <- fashion_train$fashion_labels[fashion_train$labels[num]+1]
image(t(apply(test, 2, rev)), col = gray.colors(256), axes = FALSE,
      main = item_name)
```



```
print(item_name)
```

```
[1] "Pullover"
```

```
img_rows <- 28L; img_cols <- 28L; channels <- 1L
x_train <- array_reshape(x_train, c(nrow(x_train), img_rows, img_cols, channels)) # Note: onl
x_test  <- array_reshape(x_test,  c(nrow(x_test),  img_rows, img_cols, channels))
# x_train <- x_train / 255 # normalize by dividing by max-value (1=most dark; 0=most bright)
# x_test  <- x_test  / 255
y_train_cat <- to_categorical(y_train, num_classes = 10)  # one-hot targets
y_test_cat  <- to_categorical(y_test,  num_classes = 10)
```

```
k_clear_session()
cnn <- keras_model_sequential() |>
  layer_conv_2d(filters = 8, kernel_size = c(3,3), activation = "relu",
                input_shape = c(img_rows, img_cols, channels)) |>
  layer_max_pooling_2d(pool_size = c(2,2)) |>
  layer_flatten() |>
  # layer_dropout(rate = 0.95) |>
  layer_dense(units = 8, activation = "relu") |>
  layer_dense(units = 10, activation = "softmax")
```

```r
set.seed(1984)
cnn |>
  compile(
    optimizer = optimizer_adam(),
    loss      = "sparse_categorical_crossentropy",
    metrics   = "accuracy"
  )

history_cnn <- cnn |>
  fit(
    x_train, y_train,
    epochs = 3,
    batch_size = 128,
    validation_split = 0.1,
    verbose = 0
  )

# Evaluate on test set
cnn_eval <- cnn |>
  evaluate(x_test, y_test, verbose = 0)
cat(sprintf("\n[fashion] CNN - test accuracy: %.3f\n", as.numeric(cnn_eval['accuracy'])))
```

```
[fashion] CNN - test accuracy: 0.834
```

The accuracy is 0.834.

Increasing M (number of hidden layers), K (number of convolutional layers), the number of filters, or the number of hidden units in the fully connected layer should improve performance, as it allows the model to capture more local features.

3.

```r
k_clear_session()
cnn_complex <- keras_model_sequential() |>
  layer_conv_2d(filters = 32, kernel_size = c(3,3), activation = "relu",
                input_shape = c(img_rows, img_cols, channels)) |>
  layer_max_pooling_2d(pool_size = c(2,2)) |>
  layer_conv_2d(filters = 64, kernel_size = c(3,3), activation = "relu",
                input_shape = c(img_rows, img_cols, channels)) |>
```

```
  layer_max_pooling_2d(pool_size = c(2,2)) |>
  layer_flatten() |>
  # layer_dropout(rate = 0.95) |>
  layer_dense(units = 64, activation = "relu") |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 10, activation = "softmax")

cnn_complex |>
  compile(
    optimizer = optimizer_adam(),
    loss      = "sparse_categorical_crossentropy",
    metrics   = "accuracy"
  )

history_cnn_complex <- cnn_complex |>
  fit(
    x_train, y_train,
    epochs = 3,
    batch_size = 128,
    validation_split = 0.1,
    verbose = 0
  )

# Evaluate on test set
cnn_complex_eval <- cnn_complex |>
  evaluate(x_test, y_test, verbose = 0)
cat(sprintf("\n[fashion] cnn_complex - test accuracy: %.3f\n", as.numeric(cnn_complex_eval['a
```

```
[fashion] cnn_complex - test accuracy: 0.854
```

The accuracy decreased, likely due to overfitting, as no regularization was applied.

From the perspective of the bias-variance trade-off, the model achieved lower bias, but the variance increased too much, which led to the drop in performance.

```
cnn |> summary()
```

```
Model: "sequential"

 Layer (type)                        Output Shape                    Param #
```

```
 conv2d (Conv2D)                     (None, 26, 26, 8)                80

 max_pooling2d (MaxPooling2D)        (None, 13, 13, 8)                 0

 flatten (Flatten)                   (None, 1352)                      0

 dense (Dense)                       (None, 8)                    10,824

 dense_1 (Dense)                     (None, 10)                       90

Total params: 32,984 (128.85 KB)
Trainable params: 10,994 (42.95 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 21,990 (85.90 KB)
```

```
cnn_complex |> summary()
```

```
Model: "sequential"

 Layer (type)                        Output Shape                     Param #

 conv2d (Conv2D)                     (None, 26, 26, 32)                 320

 max_pooling2d (MaxPooling2D)        (None, 13, 13, 32)                   0

 conv2d_1 (Conv2D)                   (None, 11, 11, 64)              18,496

 max_pooling2d_1 (MaxPooling2D)      (None, 5, 5, 64)                     0

 flatten (Flatten)                   (None, 1600)                         0

 dense (Dense)                       (None, 64)                     102,464

 dense_1 (Dense)                     (None, 32)                       2,080

 dense_2 (Dense)                     (None, 10)                         330

Total params: 371,072 (1.42 MB)
Trainable params: 123,690 (483.16 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 247,382 (966.34 KB)
```

4.

Because downsampling increases bias and reduces variance. However, if the downsampling is too aggressive, too much detail will be lost, making it difficult for the filters to effectively capture features.

5.

```
predictions <- cnn %>% predict(x_test)
```

```
289/289 - 0s - 723us/step
```

```
predicted_classes <- apply(predictions, 1, which.max)- 1
accuracy_by_class_dt <- data.table(actual=y_test,predicted=predicted_classes)
accuracy_by_class_dt[,correct := fifelse(actual==predicted, yes = 1, no = 0)]
fashion_labels <- c("T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
"Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot")
fashion_labels_dt <- data.table(label=fashion_labels,number=0:9)
accuracy_by_class_dt <- merge(x=accuracy_by_class_dt,y=fashion_labels_dt,by.x='actual',by.y=
accuracy_by_class_dt[,.(accuracy=mean(correct)),by=label][order(accuracy,decreasing = T)]
```

	label	accuracy
	<char>	<num>
1:	Trouser	0.9568106
2:	Bag	0.9549738
3:	Ankle boot	0.9521219
4:	Sandal	0.9353779
5:	Dress	0.8861439
6:	Sneaker	0.8807242
7:	T-shirt/top	0.8276256
8:	Coat	0.8088843
9:	Pullover	0.6640538
10:	Shirt	0.4793213

From the table above, we can see that there are significant differences in predictability across categories. For example, Ankle boot, Sandal, and Trouser are the easiest to classify correctly, whereas Shirt is difficult to classify.

This can be considered reasonable.

6.

```
fashion_unlabeled <- readRDS("~/Documents/ML/Labs/Lab3/fashion_2015_2016_unlabeled.rds")

x_test <- fashion_unlabeled$images

predictions <- cnn %>% predict(x_test)
```

```
2188/2188 - 1s - 577us/step
```

```
predicted_classes <- apply(predictions, 1, which.max)- 1
fashion_unlabeled$labels <- predicted_classes

demo_dt <- as.data.table(fashion_unlabeled$demographics)
demo_dt[, id := .I]

labels_dt <- data.table(id = seq_along(fashion_unlabeled$labels),
                        labels = fashion_unlabeled$labels)

merged_dt <- merge(demo_dt, labels_dt, by = "id")

fashion_labels_dt
```

```
          label number
         <char>  <int>
 1: T-shirt/top      0
 2:     Trouser      1
 3:    Pullover      2
 4:       Dress      3
 5:        Coat      4
 6:      Sandal      5
 7:       Shirt      6
 8:     Sneaker      7
 9:         Bag      8
10:  Ankle boot      9
```

```
fashion_full_dt <- merge(
  x    = merged_dt,
  y    = fashion_labels_dt,
  by.x = "labels",
```

```
  by.y = "number",
  all.x = TRUE
)
```

```
fashion_full_dt
```

```
Key: <labels>
       labels    id   age income education_years urban region   ses_score  year
        <int> <int> <num>  <num>           <num> <int>  <int>       <num> <num>
    1:      0    12    23  15000              13     1      4 -1.74396981  2016
    2:      0    18    18  38508              12     0      4 -1.27272667  2016
    3:      0    21    33  15000              20     1      2 -1.17068132  2016
    4:      0    29    27  46549              16     1      1 -0.20636911  2016
    5:      0    30    26  15696              16     1      2 -1.46977411  2016
   ---
69996:      9 69920    66  72640              15     0      2  0.35258376  2017
69997:      9 69929    26  55851              12     0      1 -0.57767927  2017
69998:      9 69941    42  31992              17     1      2 -0.73520637  2017
69999:      9 69960    42  60341              17     0      3  0.01381273  2017
70000:      9 69963    32  51129              14     1      4 -0.19894478  2017
            label
           <char>
    1: T-shirt/top
    2: T-shirt/top
    3: T-shirt/top
    4: T-shirt/top
    5: T-shirt/top
   ---
69996:  Ankle boot
69997:  Ankle boot
69998:  Ankle boot
69999:  Ankle boot
70000:  Ankle boot
```

```
fashion_full_dt |>
  group_by(label, year) |>
  summarize(
    age_average = mean(age),
    income_average = mean(income),
    education_years_average = mean(education_years)
  )
```

```
`summarise()` has grouped output by 'label'. You can override using the
`.groups` argument.
```

```
# A tibble: 20 x 5
# Groups:   label [10]
   label        year age_average income_average education_years_average
   <chr>       <dbl>       <dbl>          <dbl>                   <dbl>
 1 Ankle boot   2016        35.1         71200.                    15.6
 2 Ankle boot   2017        35.2         72093.                    15.6
 3 Bag          2016        35.4         77041.                    15.3
 4 Bag          2017        35.3         77007.                    15.3
 5 Coat         2016        39.3         64350.                    14.2
 6 Coat         2017        39.0         64935.                    14.2
 7 Dress        2016        39.0         54204.                    13.8
 8 Dress        2017        39.6         54305.                    13.9
 9 Pullover     2016        35.8         53674.                    13.8
10 Pullover     2017        35.9         55097.                    13.8
11 Sandal       2016        35.0         60380.                    14.5
12 Sandal       2017        35.1         61351.                    14.5
13 Shirt        2016        35.9         60109.                    14.4
14 Shirt        2017        35.9         58704.                    14.4
15 Sneaker      2016        31.2         65868.                    15.1
16 Sneaker      2017        31.6         66233.                    15.1
17 T-shirt/top  2016        35.5         49967.                    13.4
18 T-shirt/top  2017        35.6         49559.                    13.4
19 Trouser      2016        35.4         48617.                    13.4
20 Trouser      2017        35.3         48546.                    13.4
```

```
fashion_full_dt$label <- factor(fashion_full_dt$label)
levels(fashion_full_dt$label)
```

```
 [1] "Ankle boot"  "Bag"         "Coat"        "Dress"       "Pullover"
 [6] "Sandal"      "Shirt"       "Sneaker"     "T-shirt/top" "Trouser"
```

```
fashion_full_dt$label <- relevel(fashion_full_dt$label, ref = "Bag")

mn_fit <- multinom(label ~ age + year + ses_score, data = fashion_full_dt, trace = FALSE)


summary(mn_fit)
```

```
Call:
multinom(formula = label ~ age + year + ses_score, data = fashion_full_dt,
    trace = FALSE)

Coefficients:
            (Intercept)          age         year  ses_score
Ankle boot    31.582456 -0.001350428 -0.015488906 -0.4158724
Coat          70.540906  0.029539005 -0.035270475 -0.7686432
Dress         15.229378  0.030398837 -0.007865922 -1.3656654
Pullover      23.378161  0.004931656 -0.011558961 -1.4395900
Sandal       -27.655063 -0.001486307  0.013956283 -1.0561756
Shirt         25.797166  0.005247739 -0.012773021 -1.1004111
Sneaker        6.850798 -0.033390436 -0.002666012 -0.7309371
T-shirt/top -106.532065  0.002898893  0.052941428 -1.7547023
Trouser      -49.578806  0.001452963  0.024648619 -1.8397670

Std. Errors:
             (Intercept)         age         year  ses_score
Ankle boot  9.996743e-08 0.001493200 2.763161e-05 0.01296506
Coat        7.043581e-08 0.001457946 2.808786e-05 0.01211387
Dress       4.022665e-08 0.001502953 2.900902e-05 0.01205597
Pullover    4.407765e-08 0.001609769 2.994528e-05 0.01326501
Sandal      6.069024e-08 0.001544146 2.833085e-05 0.01250666
Shirt       6.189601e-08 0.001617627 3.003053e-05 0.01362205
Sneaker     8.154827e-08 0.001606065 2.794815e-05 0.01299275
T-shirt/top 3.766376e-08 0.001561360 2.907777e-05 0.01257257
Trouser     3.386542e-08 0.001603985 2.988507e-05 0.01322837

Residual Deviance: 303278.6
AIC: 303350.6
```

```
exp(coef(mn_fit))
```

```
             (Intercept)       age      year ses_score
Ankle boot  5.200996e+13 0.9986505 0.9846304 0.6597645
Coat        4.320422e+30 1.0299796 0.9653443 0.4636417
Dress       4.111827e+06 1.0308656 0.9921649 0.2552108
Pullover    1.422350e+10 1.0049438 0.9885076 0.2370249
Sandal      9.762452e-13 0.9985148 1.0140541 0.3477833
Shirt       1.597963e+11 1.0052615 0.9873082 0.3327343
Sneaker     9.446344e+02 0.9671609 0.9973375 0.4814576
T-shirt/top 5.416417e-47 1.0029031 1.0543679 0.1729587
```

```
Trouser      2.938990e-22 1.0014540 1.0249549 0.1588544
```

In general, as the ses_score increases, the probability of choosing a certain product rather than a bag decreases. For example, for ankle boots, each one-unit increase in ses_score reduces the relative odds ratio of choosing ankle boots over bags to 66% of its original value.

On the other hand, age and year have little impact on product choice, as their relative odds ratios for choosing a particular product over bags remain largely unchanged.